

Looking forward to hearing from you.

Best regards,
Girish Kumbhar
Pune
+91 7219130700



Yesterday 7:13 PM

can u just let me know what is this assignment what to do with this

Python/Django Hiring Assignment: AI-Assisted Box Selection System

Context

We operate an ecommerce platform. When a customer places an order, the warehouse team needs to know which shipping box should be used. Each product has dimensions and

Show more ▾

Yes. In simple terms, **they want you to build a small Django backend/API that takes an order's products and recommends the cheapest/smallest suitable shipping box.**

What you actually need to build

Think of it like this:

Input:

Order

- └─ Product A → 10 × 8 × 5 cm, 1 kg
- └─ Product B → 15 × 10 × 5 cm, 2 kg

Available boxes:

- Box 1 → 20 × 15 × 10 cm, max 5 kg, cost ₹50
- Box 2 → 30 × 20 × 15 cm, max 10 kg, cost ₹80

Your Django system should calculate:

"Which box can actually fit all these products and handle their total weight?"

And return something like:

JSON

```
{  
  "recommended_box": "Box 1",  
  "cost": 50  
}
```

What they probably expect

1. Django project

Create a Django project with models such as:

Product

```
name  
length  
width  
height  
weight
```

Box

```
name  
internal_length  
internal_width  
internal_height  
max_weight  
cost
```

Then you need an **order/selection API**.

For example:

http

POST /api/recommend-box/

with:

JSON

```
{
  "products": [
    {
      "id": 1,
      "quantity": 2
    },
    {
      "id": 3,
      "quantity": 1
    }
  ]
}
```

And your API returns the recommended box.

2. The important part: Box-selection algorithm

This is actually the **main part of the assignment**.

You need to decide:

Weight

Calculate total weight:

$\text{product weight} \times \text{quantity}$

Example:

Product A = 2 kg $\times$ 2 = 4 kg

Product B = 1 kg $\times$ 1 = 1 kg

Total = 5 kg

The box must have:

$\text{max_weight} \geq 5 \text{ kg}$

Dimensions

You also need to determine whether the products can fit inside the box.

This is where you need to show **your own thinking**.

For example, you could start with a simplified assumption:

Products can be rotated, but cannot overlap, and all products must fit within the internal dimensions of one box.

You then need to explain your approach in the README.

3. Database

You should probably create seed data so the reviewer can immediately run the project.

For example:

Products

Laptop	30×20×3	2kg
Book	25×18×3	0.5kg
Headphones	20×15×10	0.5kg

and:

Boxes

Small Box	35×25×10	5kg	₹40
Medium Box	45×35×20	10kg	₹70
Large Box	60×40×30	20kg	₹100

4. Test cases

They specifically ask for test cases.

You should test things like:

Test 1 — Fits in smallest box

Products → small

Expected → Small Box

Test 2 — Weight exceeds box

Product total = 8kg

Box capacity = 5kg

Expected → Box rejected

Test 3 — Dimensions don't fit

Product = 50cm long

Box = 40cm long

Expected → Box rejected

Test 4 — No box available

Product = very large/heavy

Expected:

No suitable box found

Test 5 — Multiple quantities

Product A × 3

Product B × 2

Expected → correct combined weight/dimension handling

5. README.md

Explain:

Project overview

Tech stack

How to install

How to run

Database setup

API endpoints

Request/response examples

Box-selection logic

Assumptions

How to run tests

6. AI_USAGE.md

This is **very important** because they're explicitly allowing AI.

They want you to document things like:

AI Tool: ChatGPT

Prompt:

"Help me understand how to structure a Django API..."

Output accepted:

...

Output rejected:

...

Why I rejected it:

...

Mistake found:

AI suggested X, but I found that...

How I verified:

I wrote tests / manually tested API / checked Django documentation...

Don't fake this file. Keep your actual prompts and explain what you changed.

7. Chat transcript

This is unusual and **very important**.

They explicitly say:

"Export the chat transcript and attach and include in your repository."

And:

"Make sure that you don't generate this using AI."

So **don't ask ChatGPT to manufacture a fake conversation transcript**.

Keep/export the actual conversation you have with the AI tools you use.

8. "What did you learn?"

They specifically say **not to use AI to generate this answer**.

So you'll need to write this yourself.

For example, after actually doing the assignment, you might write your own thoughts about:

- Django models
- REST APIs
- box-selection logic
- testing
- edge cases
- limitations of your algorithm
- how you used AI and verified it

But **don't copy an AI-generated answer** for this section.

9. TEST_OUTPUT.md

Run your tests:

Bash

```
python manage.py test
```

Then paste the actual terminal output:

```
Creating test database...
```

```
.....
```

```
-----  
Ran 5 tests in 0.XXXs
```

```
OK
```

That's what they mean by **test run output**.

So your final GitHub repository could look like

```
box-selection-system/  
|  
├─ manage.py  
├─ requirements.txt  
├─ README.md  
├─ AI_USAGE.md  
├─ TEST_OUTPUT.md  
|  
├─ products/  
|   └─ models.py
```

```
| | views.py
| | serializers.py
| | urls.py
| | tests.py
| | ...
|
| box_selector/
| | settings.py
| | urls.py
| | ...
|
└─ chat-transcript/
    └─ actual-ai-chat.txt
```

In one sentence:

They are testing whether you can build a small Django application + design a reasonable packing algorithm + create APIs + write tests + explain your engineering decisions, while being transparent about AI assistance.

And because they specifically say "**show your own thinking**," I would **not blindly ask AI to build the entire assignment and submit it**. Use AI as a coding assistant, but understand and test every important piece—especially the **box-selection algorithm**.





 Free offer



Yes. Since the **Django product is working and all 10 tests pass**, you're at the submission stage. 🍊

For this assignment, don't just upload the code. Submit a **clean GitHub repository** containing the code + required documentation + test evidence.

1. Your final repository should look like this

```
box-selection-system/
|
├─ config/
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   ├── asgi.py
|   └── wsgi.py
|
├─ packaging/
|   ├── migrations/
|   ├── __init__.py
|   ├── admin.py
|   ├── models.py
|   ├── serializers.py
|   ├── services.py
|   ├── tests.py
|   ├── urls.py
|   └── views.py
|
├─ .github/
|   └── workflows/
|       └── tests.yml
|
├─ .gitignore
├─ AI_USAGE.md
├─ README.md
├─ TEST_OUTPUT.md
├─ manage.py
├─ requirements.txt
└─ db.sqlite3      ❌ don't commit this
```

The important assignment files are:

```
README.md
AI_USAGE.md
TEST_OUTPUT.md
.github/workflows/tests.yml
```

And of course the actual Django application.

2. First create .gitignore

Create:

```
.gitignore
```

Put:

```
gitignore

# Python
__pycache__/
*.py[cod]
*.pyo

# Virtual environment
.venv/
venv/
env/

# Django
db.sqlite3
*.log

# Environment variables
.env
.env.*

# macOS
.DS_Store

# IDE
.vscode/
.idea/
```

This is important because you **should not push** `.venv` or your local SQLite database.

3. Make sure requirements.txt is correct

Run:

```
Bash

cat requirements.txt
```

It should contain Django and DRF.

For example:

```
Django==6.1.0
djangorestframework==3.16.1
```

Your exact installed versions may differ, so don't blindly replace them.

You can get your actual versions with:

Bash

```
pip freeze
```

For this assignment, I'd keep `requirements.txt` focused on the packages your project actually needs rather than dumping every environment package.

4. Create TEST_OUTPUT.md

You already have excellent test evidence:

```
Found 10 test(s).
```

```
.....
```

```
-----
Ran 10 tests in 0.010s
```

```
OK
```

Create:

```
TEST_OUTPUT.md
```

Put:

Markdown

```
# Test Output
```

```
Tests were executed using:
```

```
```bash
```

```
python manage.py test
```

Result:

```
Found 10 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
```

```
.....
```

```

Ran 10 tests in 0.010s
```

```
OK
```

```
Destroying test database for alias 'default'...
```

All 10 tests passed successfully.

```

```

```
5. Create GitHub Actions
```

This is worth doing because the assignment specifically asks for test-run output

Create:

```
```text
.github/workflows/tests.yml
```

Use:

YAML

```
name: Django Tests
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
  pull_request:
```

```
    branches:
```

```
      - main
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout repository
```

```
        uses: actions/checkout@v4
```

```
      - name: Set up Python
```

```
        uses: actions/setup-python@v5
```

```
        with:
```

```
          python-version: "3.13"
```

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt

- name: Run migrations
  run: python manage.py migrate

- name: Run tests
  run: python manage.py test
```

Then whenever you push to GitHub, GitHub will automatically run your tests.

6. Create the README

This is probably the **most important file for the evaluator**.

Your `README.md` should explain:

Project

AI-Assisted Box Selection System

What it does

Explain:

- Products have dimensions and weight.
- Boxes have internal dimensions, maximum weight and cost.
- API accepts products + quantities.
- System checks weight constraints.
- System checks whether products can physically fit.
- Products can be rotated.
- It recommends the **lowest-cost suitable box**.

Tech stack

Python
Django
Django REST Framework
SQLite

Setup

Bash

```
git clone <your-repository-url>
cd box-selection-system

python3 -m venv .venv
source .venv/bin/activate

pip install -r requirements.txt

python manage.py migrate
python manage.py runserver
```

API

POST /api/recommend-box/

Example request:

JSON

```
{
  "products": [
    {
      "product_id": 1,
      "quantity": 1
    },
    {
      "product_id": 2,
      "quantity": 1
    }
  ]
}
```

Example response:

JSON

```
{
  "recommended_box": {
    "id": 2,
    "name": "Medium Box",
    "length": 50.0,
    "width": 35.0,
    "height": 20.0,
    "max_weight": 10.0,
    "cost": 80.0
  }
}
```

```
}  
}
```

Also document:

```
200 → suitable box found  
400 → invalid request/product/quantity  
404 → no suitable box found
```

7. Explain your packing algorithm

This part is particularly important.

Don't claim that you've implemented a mathematically optimal 3D bin-packing solver.

Your implementation is a **heuristic 3D packing approach**.

Explain something like:

The system uses a heuristic 3D packing algorithm. Products are sorted by volume, and each product is attempted in available free spaces. All unique rotations of each product are considered. After placing a product, the remaining space is split into usable regions. A box is considered suitable only when all products can be placed and the total weight is within the box's maximum weight.

Then state:

The algorithm is designed as a practical heuristic rather than an exact optimal bin-packing solver.

That is actually a **good engineering answer** because you're being honest about the algorithm's limitations.

8. Create AI_USAGE.md

This is **mandatory according to your assignment**.

And this is where you should be careful.

The assignment asks for:

- AI tools used
- prompts

- accepted output
- rejected/modified output
- AI mistakes
- how you verified the output

You can document the actual work we did.

For example:

Markdown

AI Usage

AI Tool Used

ChatGPT was used as a development assistant during the assignment.

Areas Where AI Was Used

AI assistance was used for:

- Django project setup guidance
- Django REST Framework API structure
- Serializer design
- API validation
- Initial implementation ideas for the box recommendation algorithm
- Test case suggestions
- Debugging test failures
- README/documentation structure
- GitHub Actions configuration

Important Prompts / Questions

Examples of prompts used during development:

- How should I structure a small Django REST API for this assignment?
- How should I model products and boxes in Django?
- How can I validate the product quantity in Django REST Framework?
- How should I implement a box recommendation algorithm that considers dimensions, r
- How should I test the recommendation API?
- Why is my quantity test failing?
- How should I prepare the Django project for GitHub submission?

Accepted AI Output

AI-generated suggestions were used for parts of the Django API structure, serializer

All generated code was reviewed, executed and tested locally before being retained.

Modified / Rejected Output

Some generated suggestions were modified after testing.

For example, an initial test expected two laptops to require a Medium Box. After imp

The test expectation was therefore corrected rather than changing the algorithm to s

AI Mistakes / Limitations

One important issue identified during verification was that an initial test assumptio

The AI-assisted implementation also uses a heuristic packing strategy rather than an

Verification

AI-generated code was verified by:

- Running Django system checks
- Running the Django test suite
- Manually testing the API with curl
- Checking expected HTTP status codes
- Checking product rotation
- Checking quantity handling
- Checking box weight limits
- Checking the no-suitable-box scenario

Final test result:

10 tests passed successfully.

However: don't blindly copy that whole section if it doesn't accurately represent your actual usage. The assignment specifically asks for honesty.

9. VERY IMPORTANT — "What did you learn?"

The assignment says:

"What did you learn?" must NOT be AI-generated.

So do not ask me to write this section for you and don't copy an AI-written paragraph.

Write it yourself in your own words.

It can be simple.

For example, you might personally mention things you actually learned while doing the assignment:

- getting back into Django
- Django REST Framework serializers
- test-driven debugging
- 3D packing constraints
- product rotation
- handling quantities
- using GitHub Actions

But **you** should write the actual paragraph yourself.

10. Before pushing, run these commands

From your project root:

Bash

```
python manage.py check
```

Then:

Bash

```
python manage.py test
```

Then:

Bash

```
git status
```

Make sure you **do NOT** see:

```
.venv/  
db.sqlite3  
__pycache__/
```

11. Initialize Git

If you haven't already:

Bash

```
git init
```

Then:

```
Bash
```

```
git add .
```

Check:

```
Bash
```

```
git status
```

You should see your source files and documentation staged.

Then:

```
Bash
```

```
git commit -m "Initial Django box selection system"
```

12. Create GitHub repository

On GitHub create a repository, for example:

```
box-selection-system
```

I recommend **don't initialize it with another README**, since you already have one locally.

Then connect your local project:

```
Bash
```

```
git remote add origin YOUR_GITHUB_REPOSITORY_URL
```

Check:

```
Bash
```

```
git remote -v
```

Then:

Bash

```
git branch -M main
git push -u origin main
```

13. Check GitHub after pushing

Your GitHub repository should show something like:

box-selection-system

```
|— .github/
|— config/
|— packaging/
|— .gitignore
|— AI_USAGE.md
|— README.md
|— TEST_OUTPUT.md
|— manage.py
|— requirements.txt
```

And GitHub Actions should show:

 Django Tests — Passed

14. Don't forget the chat transcript

This is a **special requirement** in your assignment.

They explicitly said the chat transcript should be exported/included and that:

Transcript must NOT be generated by AI.

So **don't let me create a fake transcript for you.**

Use the actual ChatGPT conversation/export mechanism and include the genuine transcript as requested by the assignment.

If they specifically require a file inside the repository, name it something like:

CHAT_TRANSCRIPT.pdf

or whatever format their instructions specify.

15. Final submission checklist

Before submitting, check this:

Requirement	Status
Django project	☒
Product model	☒
Box model	☒
REST API	☒
Product quantity	☒
Product rotation	☒
Weight constraint	☒
Box recommendation	☒
10 automated tests	☒
Tests passing	☒
README	☐
AI_USAGE.md	☐
TEST_OUTPUT.md	☐
GitHub Actions	☐
.gitignore	☐
Actual chat transcript	☐
Your own "What did you learn?"	☐
GitHub repository	☐

One thing I would not do yet

Don't push the repository immediately.

Let's first make the repository **submission-quality**, especially `README.md` , `AI_USAGE.md` , GitHub Actions, and the final code review.

If you want, we can go **one step at a time from here**, starting with `.gitignore` and then I'll give you the exact content for each required file.



Today 5:24 PM

make the readme file for me

Absolutely. Since your project is now working and the 10 tests pass, here is a **submission-ready** `README.md` you can directly copy.

Markdown

```
# AI-Assisted Box Selection System
```

A Django REST API that recommends the most suitable shipping box for a given set of |

The system considers:

- Product dimensions
- Product weight
- Product quantity
- Box internal dimensions
- Box maximum weight capacity
- Product rotation/orientation
- Box cost

The system recommends the **lowest-cost** box that can physically contain all requested

```
## Tech Stack
```

- Python 3.13
- Django
- Django REST Framework
- SQLite
- GitHub Actions

```
## Project Structure
```

```
```text
```

```
box-selection-system/
|
├─ config/
| ├── settings.py
| ├── urls.py
| ├── asgi.py
| └── wsgi.py
|
├─ packaging/
| ├── migrations/
| ├── admin.py
| ├── models.py
| ├── serializers.py
| ├── services.py
| ├── tests.py
| ├── urls.py
| └── views.py
|
├─ .github/
| └── workflows/
| └── tests.yml
|
├─ .gitignore
├─ AI_USAGE.md
├─ README.md
├─ TEST_OUTPUT.md
├─ manage.py
└─ requirements.txt
```

## Features

### Product Management

Each product contains:

- Name
- Length
- Width
- Height
- Weight

### Box Management

Each box contains:

- Name
- Internal length
- Internal width

- Internal height
- Maximum supported weight
- Cost

Products and boxes can also be managed through Django Admin.

## Box Recommendation Logic

The recommendation process works as follows:

1. The API receives product IDs and quantities.
2. The system retrieves the corresponding products from the database.
3. Product quantities are expanded into individual items for packing.
4. Total product weight is calculated.
5. Boxes that cannot support the total weight are discarded.
6. Remaining boxes are checked using a 3D packing heuristic.
7. Products can be rotated into different orientations.
8. Products are attempted to be placed into available spaces inside the box.
9. Boxes where all products can be placed are considered suitable.
10. The suitable boxes are sorted by cost.
11. The cheapest suitable box is returned.

The packing algorithm is a **heuristic 3D packing approach**, not an exact optimal bin-packing solver.

## API

### Recommend Box

#### Endpoint

POST /api/recommend-box/

#### Request

JSON

```
{
 "products": [
 {
 "product_id": 1,
 "quantity": 1
 }
]
}
```

```
 },
 {
 "product_id": 2,
 "quantity": 1
 }
]
}
```

Request Fields

Field	Type	Description
products	Array	List of products to pack
product_id	Integer	ID of the product
quantity	Integer	Number of units required

quantity must be at least 1 .

At least one product is required.

Successful Response

HTTP 200

Example:

```
JSON

{
 "recommended_box": {
 "id": 2,
 "name": "Medium Box",
 "length": 50.0,
 "width": 35.0,
 "height": 20.0,
 "max_weight": 10.0,
 "cost": 80.0
 }
}
```

Error Responses

## Invalid Request

### HTTP 400

Returned when the request contains invalid data.

Examples:

- Missing products
- Empty product list
- Quantity less than 1
- Invalid product ID

Example:

JSON

```
{
 "error": {
 "products": [
 "At least one product is required."
]
 }
}
```

## Product Not Found

### HTTP 400

Example:

JSON

```
{
 "error": "Product with id 99999 does not exist."
}
```

## No Suitable Box

### HTTP 404

Returned when no available box can contain all requested products while satisfying the weight limit.

Example:

JSON

```
{
 "message": "No suitable box found"
}
```

## Example Data

The project was tested with sample products and boxes.

### Products

Product	Length	Width	Height	Weight
Laptop	30	20	5	2
Keyboard	45	15	5	1
Heavy Machine	20	20	20	15

### Boxes

Box	Length	Width	Height	Max Weight	Cost
Small Box	35	25	10	5	50
Medium Box	50	35	20	10	80
Large Box	70	50	40	20	120

## Running the Project Locally

### 1. Clone the Repository

Bash

```
git clone <YOUR_GITHUB_REPOSITORY_URL>
```

Move into the project directory:

Bash

```
cd box-selection-system
```

## 2. Create Virtual Environment

Bash

```
python3 -m venv .venv
```

Activate it on macOS/Linux:

Bash

```
source .venv/bin/activate
```

On Windows:

Bash

```
.venv\Scripts\activate
```

## 3. Install Dependencies

Bash

```
pip install -r requirements.txt
```

## 4. Run Database Migrations

Bash

```
python manage.py migrate
```

## 5. Create Admin User

Bash

```
python manage.py createsuperuser
```

## 6. Start the Development Server

Bash

```
python manage.py runserver
```

The application will be available at:

```
http://127.0.0.1:8000/
```

## Django Admin

Django Admin is available at:

```
http://127.0.0.1:8000/admin/
```

Products and boxes can be created and managed from the admin interface.

## Testing

The project contains automated API tests covering the main recommendation scenarios.

Run the tests using:

Bash

```
python manage.py test
```

Current test suite:

```
Found 10 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
```

```
.....
```

```

Ran 10 tests in 0.010s
```

```
OK
```

```
Destroying test database for alias 'default'...
```

## Test Cases Covered

The test suite verifies:

1. Single product recommendation
2. Multiple product recommendation
3. Product quantity handling
4. Product rotation
5. Box weight limits
6. No suitable box scenario
7. Invalid product ID
8. Invalid quantity
9. Empty product list
10. Missing product list

## GitHub Actions

The project includes a GitHub Actions workflow that automatically runs the Django test suite.

Workflow:

```
.github/workflows/tests.yml
```

The workflow:

1. Checks out the repository
2. Sets up Python
3. Installs dependencies
4. Runs database migrations
5. Runs the test suite

This helps verify that the project continues to pass its automated tests after changes are pushed.

## Design Decisions

### Lowest-Cost Suitable Box

"Most suitable" is defined as the **lowest-cost box that satisfies both:**

- Physical packing constraints
- Maximum weight constraint

This makes the recommendation deterministic and cost-aware.

### Product Rotation

Products are allowed to rotate.

For example, a product with dimensions:

$10 \times 30 \times 20$

may be rotated to:

$30 \times 20 \times 10$

if that orientation allows it to fit inside the box.

### Quantity

The API accepts a quantity for each product.

For example:

JSON

```
{
 "product_id": 1,
 "quantity": 3
}
```

means three units of that product need to be packed.

Each unit is considered individually during the packing process.

## Weight

The total weight is calculated using:

```
product weight × quantity
```

A box is rejected if:

```
total product weight > box maximum weight
```

## Limitations

The packing algorithm is a heuristic rather than an exact mathematical optimization algorithm.

This means that in some complex arrangements, the algorithm may determine that a product set cannot fit even though a different packing arrangement might theoretically exist.

The implementation prioritizes:

- Simplicity
- Understandability
- Practicality
- Support for product rotation
- Quantity handling
- Weight validation

For a production-scale warehouse system, the packing algorithm could be replaced or extended with a more advanced optimization approach.

## AI Usage

AI assistance was used during development of this assignment.

Details about:

- AI tools used
- Prompts
- Accepted output
- Modified/rejected output
- AI mistakes

- Verification of AI-generated suggestions

are documented separately in:

AI\_USAGE.md

## Test Evidence

The test execution output is also documented in:

TEST\_OUTPUT.md

## Author

Girish K.

Full Stack Developer

Skills used in this project:

- Python
- Django
- Django REST Framework
- REST API development
- Database modeling
- Automated testing
- Git/GitHub

```
One important change before you commit
```

Replace this:

```
```text
<YOUR_GITHUB_REPOSITORY_URL>
```

with your actual GitHub repository URL **after you create the repository**.

Also, don't add a fake author description or claims you didn't personally make.

Your README is now enough for the evaluator to understand **what the project does, how to run it, how the API works, how the algorithm works, and how it was tested.**

Next, we should create `AI_USAGE.md`, because that is one of the assignment's mandatory files and needs to accurately document our actual AI-assisted development process.



You can export the AI chat transcript as a PDF or text file and include it in your project repository or ZIP folder. If a shareable link to the chat transcript is available, you may share that as well.

"

how to do this can u guid me step by step in this

Yes. Since the assignment specifically says the **actual AI chat transcript** can be submitted as a PDF/text file or shareable link, the safest approach is to export **this actual ChatGPT conversation**, not create a summary.

Option 1 — Export the actual ChatGPT conversation as PDF

Since you're using ChatGPT in a browser on your Mac, this is probably the easiest method.

Step 1: Open this conversation

Stay in the current ChatGPT conversation where we worked on the Django assignment.

Make sure the conversation contains the relevant work, including:

- Django setup
- Models
- API
- Packing algorithm
- Tests
- Debugging the failed test
- Final 10/10 passing tests
- README discussion

Step 2: Use your browser's Print function

On Mac:

⌘ + P

This opens the browser's print dialog.

Step 3: Select PDF

In the print dialog, look for:

PDF → Save as PDF

Choose that.

Step 4: Name the file

I recommend:

AI_CHAT_TRANSCRIPT.pdf

Save it somewhere easy to find, for example your Downloads folder.

Step 5: Put it inside your project

Move/copy the PDF into your project root:

```
box-selection-system/  
|  
├─ config/  
├─ packaging/  
├─ .github/  
├─ README.md  
├─ AI_USAGE.md  
├─ TEST_OUTPUT.md  
├─ AI_CHAT_TRANSCRIPT.pdf ← here  
├─ manage.py  
└─ requirements.txt
```

You can do this from Terminal:

Bash

```
cd ~/path/to/box-selection-system
```

Then, if the PDF is in Downloads:

Bash

```
cp ~/Downloads/AI_CHAT_TRANSCRIPT.pdf .
```

Check:

Bash

```
ls
```

You should see:

```
AI_CHAT_TRANSCRIPT.pdf
README.md
AI_USAGE.md
TEST_OUTPUT.md
manage.py
config
packaging
...
```

Step 6: Make sure `.gitignore` isn't ignoring PDFs

Your `.gitignore` should **not** contain:

```
*.pdf
```

If it does, remove that line.

Step 7: Add the transcript to Git

Run:

```
Bash
```

```
git status
```

You should see something like:

```
Untracked files:
  AI_CHAT_TRANSCRIPT.pdf
```

Then:

```
Bash
```

```
git add AI_CHAT_TRANSCRIPT.pdf
```

And:

Bash

```
git commit -m "Add AI chat transcript"
```

Then:

Bash

```
git push
```

Step 8: Verify on GitHub

Open your GitHub repository.

You should see:

```
box-selection-system
|
├─ .github
├─ config
├─ packaging
├─ AI_CHAT_TRANSCRIPT.pdf
├─ AI_USAGE.md
├─ README.md
├─ TEST_OUTPUT.md
├─ .gitignore
├─ manage.py
└─ requirements.txt
```

Clicking `AI_CHAT_TRANSCRIPT.pdf` should allow the evaluator to view the transcript.

⚠ One very important thing

Do not create the transcript yourself with something like:

```
"Here is a transcript of my conversation with ChatGPT..."
```

The assignment specifically says the transcript must be the **actual AI chat transcript**.

So exporting/printing the real conversation is the correct approach.

Option 2 — Shareable ChatGPT link

If your ChatGPT interface provides a **Share** option for this conversation, you can also use that.

Usually:

1. Open this conversation.
2. Click the **Share** button/menu.
3. Create a share link.
4. Copy the link.
5. Put it in your submission.

You can also mention it in `README.md` :

Markdown

```
## AI Chat Transcript
```

```
The actual AI-assisted development conversation is included in:
```

```
`AI_CHAT_TRANSCRIPT.pdf`
```

I would still recommend **including the PDF in the GitHub repository**, even if you also provide a share link. That way the evaluator doesn't depend on the link continuing to work.

My recommendation for your assignment

Use **both** if possible:

GitHub Repository

```
|  
├── AI_CHAT_TRANSCRIPT.pdf ← actual conversation  
├── AI_USAGE.md             ← your documentation of AI usage  
├── README.md  
└── TEST_OUTPUT.md
```

And if ChatGPT gives you a shareable link, provide that separately in the application/submission form.

Don't worry about making the PDF pretty. The important thing is that it is the genuine conversation transcript.



ChatGPT can make mistakes. Check important info.

